



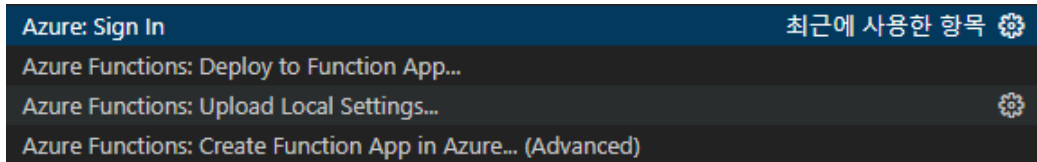
태양광 발전량 예측 실습 정리

전체 흐름도



구성 요소	이유
TimerTrigger Function (스케줄러)	매일 자동으로 데이터를 수집하기 위해 필요 (수동 아님)
Event Hub	데이터를 다른 서비스로 스트리밍 방식 전달
Stream Analytics	실시간 처리 + Power BI / Cosmos DB로 가공 데이터 분배
Cosmos DB	벡터 검색을 위한 저장소 (AI 질의응답에 핵심)
Azure OpenAI + RAG 구조	사용자 질문을 이해하고, 데이터를 기반으로 답변 생성
Gradio (또는 HTTP API)	프론트엔드 없이 바로 질의응답 테스트 가능하게 함

VSCoDe(F1키)



VSCode 작업	명령어 or 버튼	설명
Azurite 로컬 실행	<code>azurite start</code> 또는 Azure 확장 메뉴에서 "Start Azurite"	Cosmos DB 등 로컬 에뮬레이터 테스트 용 (테스트 환경에서만 필요)
Function App 로컬 테스트	<code>F5</code> (디버그 시작)	VSCode에서 HTTP 요청 테스트 가능
<code>local.settings.json</code> 설정	직접 수정	<code>환경변수</code> (KEY, ENDPOINT 등) 지정용
Azure에 배포	Azure Functions 확장 > Function App 우 클릭 > Deploy to Function App	로컬 코드를 Azure에 반영
Cosmos DB 테스트	Postman or <code>requests.post()</code> 코드	<code>chat_rag</code> , <code>solar_predict_cosmosdb</code> API 테스트

Function app.py

▼ 첫 번째 코드(`solar_predict_eventhubs_scheduler`)

목적	태양광 예측 데이터 수집 및 Event Hub 전송
Trigger 방식	Timer Trigger @app.timer_trigger
입력 데이터	CSV 파일(solar_city.csv) + 기상청 API 응답
출력 대상	Event Hub, Teams Webhook
주요 처리 로직	- 도시별 기상청 API 요청 - 발전 예측 데이터 수집 - 매일 자동 실행 및 이벤트 전송
사용 기술	KMA API + Event Hub + CSV + Teams

▼ Code

```
import logging
import azure.functions as func
import pandas as pd
import datetime
from datetime import timezone
import pytz
import requests
import json
import os

# Azure Functions 앱을 생성합니다.
```

```

app = func.FunctionApp()

# (UTC 시간대 기준) 매일 자정에 실행되는 타이머 트리거를 설정합니다.
@app.timer_trigger(schedule="0 0 0 * * *", arg_name="timer", run_on_startup=
# Event Hub에 데이터를 보내는 출력을 설정합니다.
@app.event_hub_output(arg_name="event", event_hub_name= os.environ["Sol
def solar_predict_eventhubs_scheduler(timer: func.TimerRequest, event: func.C
    # 로그를 통해 스케줄러 시작 알림을 기록합니다.
    logging.info('태양광 예측 데이터 수집 스케줄러가 시작되었습니다.')

# CSV 파일에서 태양광 데이터 읽어오기
df = pd.read_csv("./data/solar_city.csv", encoding="euc-kr")

# 현재 UTC 시간 가져오기
utc_timestamp = datetime.datetime.now(tz=timezone.utc)
# KST (한국 표준시)로 변환합니다.
kst = pytz.timezone('Asia/Seoul')
kst_timestamp = utc_timestamp.astimezone(kst)
# 현재 날짜를 "YYYYMMDD" 형식으로 변환합니다.
base_date = kst_timestamp.strftime("%Y%m%d")

# 기상 데이터 API URL 설정
url = "https://bd.kma.go.kr/kma2020/energy/energyGeneration.do"
headers = {
    "Content-Type": "application/x-www-form-urlencoded", # 요청하는 데이터 형식
}
# 수집할 태양광 데이터를 저장할 리스트입니다.
solar_data_list = []
# 데이터 수집에서 제외된 행의 인덱스를 저장할 리스트입니다.
skipped_index = []

# CSV 파일의 각 행을 반복합니다.
for index, row in df.iterrows():
    # 중요한 데이터가 없는 경우, 해당 행을 생략합니다.
    if pd.isna(row["격자X"]) or pd.isna(row["격자Y"]) or pd.isna(row["행정구역코
        # 생략된 인덱스를 리스트에 추가합니다.
        skipped_index.append(index)
        continue

    try:
        # 필요한 데이터 추출
        reg_cd = row["행정구역코드"]
        city = row["도"]

```

```

        county = row["시"]
        lat = row["위도(초/100)"]
        lon = row["경도(초/100)"]

        # API 요청에 사용할 데이터 설정
        payload = f"baseDate={base_date}&fcstDate={base_date}&fcstTime=0000"

        # API에 POST 요청을 보내고 응답을 받습니다.
        response = requests.post(url, headers=headers, data=payload)
        response.raise_for_status() # 요청이 성공하지 않으면 오류를 발생시킵니다.

        data = response.json() # JSON 형식으로 데이터를 변환합니다.

        items = data['result'] # 필요한 데이터 항목을 추출합니다.

        # 받은 데이터에서 필요한 정보만 추출하여 리스트에 추가합니다.
        for item in items:
            solar_data = {
                "city": city,
                "county": county,
                "fcstDate": item["fcstDate"],
                "fcstTime": item["fcstTime"],
                "pcap": item["pcap"],
                "qgen": item["qgen"],
                "regCd": item["regCd"],
                "srad": item["srad"],
                "temp": item["temp"],
                "wspd": item["wspd"],
                "lat": lat,
                "lon": lon
            }
            solar_data_list.append(solar_data) # 수집된 태양광 예측 데이터를 추가합니다.

    except (ValueError, TypeError) as e:
        # 데이터 변환 중 오류가 발생하면 로그를 기록합니다.
        logging.error(f"Failed to convert 격자X or 격자Y to integer in row {index}: {e}")

# 태양광 데이터 수집이 완료되었음을 기록합니다.
logging.info(f"태양광 예측 데이터 수집이 완료되었습니다. 수집된 데이터: {len(solar_data_list)}")
# 수집된 데이터를 Event Hub에 전송합니다.
event.set(solar_data_list)

# 환경 변수에서 웹훅 URL을 확인합니다.

```

```

if os.environ.get("AzureWebHookUrl") is not None:
    logging.info("환경 변수에 웹훅 URL이 설정되어있습니다. 웹훅을 요청합니다.")
    # 웹훅 URL을 불러옵니다.
    webhook_url = os.environ["AzureWebHookUrl"]
    # Teams Incoming Webhook 참고 자료: https://learn.microsoft.com/en-us/n
    # 웹훅에 보낼 데이터 형식 설정
    webhook_payload = {
        "type": "message",
        "attachments": [
            {
                "contentType": "application/vnd.microsoft.card.adaptive",
                "contentUrl": None,
                "content": {
                    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
                    "type": "AdaptiveCard",
                    "version": "1.2",
                    "body": [
                        {
                            "type": "TextBlock",
                            "text": "태양광 예측 데이터 수집 스케줄러가 실행되었습니다."
                        }
                    ]
                }
            }
        ]
    }
    # 웹훅에 POST 요청을 보내고 응답을 확인합니다.
    response = requests.post(webhook_url, json=webhook_payload)
    logging.info(f"웹훅 실행 결과: {response.status_code}")
    response.raise_for_status() # 요청 실패 시 오류를 발생시킵니다.
else:
    logging.info("웹훅 URL이 설정되어 있지 않습니다. 웹 훅 요청을 생략합니다.")

# 데이터 수집에서 제외된 행의 인덱스를 로그로 기록합니다.
logging.info(f"{skipped_index}번 행은 데이터 수집에서 제외되었습니다.")
# 종료 알림을 기록합니다.
logging.info('태양광 예측 데이터 수집 스케줄러가 종료되었습니다.')

```

▼ 두 번째 코드(`chat_rag` , `solar_predict_cosmosdb`)

목적	질의응답 처리(RAG) 및 Cosmos DB 저장
Trigger 방식	HTTP Trigger @app.route

입력 데이터	사용자가 HTTP 요청으로 보낸 JSON
출력 대상	- Cosmos DB(upset_item) - RAG 질의응답 결과 반환
주요 처리 로직	- 태양광 데이터 → 자연어 반환 - OpenAI 임베딩 생성 - CosmosDB에 벡터 저장 - 유사도 검색 & GPT 응답
사용 기술	Azure OpenAI, Cosmos DB (벡터검색), RAG

▼ Code

```
# 필요한 라이브러리 임포트
import azure.functions as func # Azure Functions 기능을 사용하기 위한 라이브러리
import logging # 로깅(실행 정보, 오류 등을 기록)을 위한 라이브러리
from openai import AzureOpenAI # Azure OpenAI 서비스를 사용하기 위한 클라이언트
import json # JSON 데이터를 처리하기 위한 라이브러리
import os # 환경 변수 접근 등 운영체제 기능을 사용하기 위한 라이브러리
from azure.cosmos import CosmosClient # Azure Cosmos DB 연결을 위한 클라이언트
from azure.cosmos.exceptions import CosmosHttpResponseError # Cosmos DB 예외
import time # 시간 지연, 대기 등의 기능을 위한 라이브러리

# 상수 정의
APPLICATION_JSON = "application/json" # HTTP 응답 형식 지정을 위한 상수

# 환경 변수에서 설정 값 로드
# Azure OpenAI 설정
openai_endpoint = os.environ["OPENAI_ENDPOINT"] # OpenAI API 엔드포인트 (URL)
openai_key = os.environ["OPENAI_KEY"] # OpenAI API 인증 키
openai_api_version = os.environ["OPENAI_API_VERSION"] # OpenAI API 버전
openai_gpt_model = os.environ["OPENAI_GPT_MODEL"] # 사용할 GPT 모델 이름
openai_embeddings_deployment = os.environ["OPENAI_EMBEDDINGS_DEPLOYMENT"] # 사용할 임베딩 배포 이름
# Azure Cosmos DB 설정
cosmos_endpoint = os.environ["COSMOSDB_ENDPOINT"] # Cosmos DB 엔드포인트 (URL)
cosmos_key = os.environ["COSMOSDB_KEY"] # Cosmos DB 접근 키
cosmosdb_database = os.environ["COSMOSDB_DATABASE"] # 사용할 데이터베이스 이름
cosmosdb_container = os.environ["COSMOSDB_CONTAINER"] # 사용할 컨테이너 이름

# OpenAI 클라이언트 초기화 - AI 모델을 호출하기 위한 인터페이스
openai_client = AzureOpenAI(
    azure_endpoint=openai_endpoint,
    api_key=openai_key,
    api_version=openai_api_version
)
```

```

# Cosmos DB 클라이언트 초기화 - 데이터베이스 연결 설정
cosmos_client = CosmosClient(cosmos_endpoint, cosmos_key)
database = cosmos_client.get_database_client(cosmosdb_database)
container = database.get_container_client(cosmosdb_container)

# 헬퍼 함수 정의
def solar_data_to_text(item):
    """ 태양광 데이터를 자연어 텍스트로 변환하는 함수
    매개변수:
        item (dict): 태양광 발전 데이터가 담긴 딕셔너리
    반환값:
        str: 데이터를 설명하는 자연어 텍스트
    """

    # 소수점 자릿수 제한
    pcap = round(float(item['pcap']), 2)
    qgen = round(float(item['qgen']), 2)
    usage_rate = round((qgen / pcap * 100), 2) if pcap > 0 else 0

    text = f"【위치】 {item['city']} {item['county']} (위도 {round(float(item['lat']), 2}
    text += f"【날짜/시간】 {item['fcstDate']} {item['fcstTime']}시\n"
    text += f"【발전 정보】 설비용량: {pcap}kW | 발전량: {qgen}kW | 이용률: {usage_rate}%
    text += f"【환경 조건】 일사량: {item['srad']}W/m² | 온도: {item['temp']}°C | 풍속

    # RAG 최적화를 위한 예상 질문 추가
    text += f"\nQ: {item['city']} {item['county']}의 {item['fcstDate'][:4]}년 {item['fcstTime']}시
    text += f"A: 설비용량 {pcap}kW에서 {qgen}kW 발전하여 이용률은 {usage_rate}%

    return text

def batch_generate_embeddings(texts, batch_size=20):
    """ 텍스트 배열에 대해 배치 단위로 임베딩(벡터화)을 생성하는 함수

    매개변수:
        texts (list): 임베딩할 텍스트 목록
        batch_size (int): 한 번에 처리할 텍스트 수(기본값: 20)

    반환값:
        list: 생성된 임베딩 벡터 목록
    """
    all_embeddings = []
    # 배치 크기만큼 나누어 처리

```

```

for i in range(0, len(texts), batch_size):
    batch = texts[i:i + batch_size]
    try:
        # OpenAI API를 호출하여 임베딩 생성
        response = openai_client.embeddings.create(
            input=batch,
            model=openai_embeddings_deployment
        )
        embeddings_data = response.model_dump()
        batch_embeddings = [data['embedding'] for data in embeddings_data['data']]
        all_embeddings.extend(batch_embeddings)
        logging.info(f"임베딩 배치 처리 완료: {i} ~ {i + len(batch)} / {len(texts)}")
    except Exception as e:
        logging.error(f"임베딩 생성 오류 (배치 {i}-{i+len(batch)}): {str(e)}")
        # 오류 발생 시 빈 임베딩으로 채우기 (1536은 일반적인 OpenAI 임베딩 차원)
        all_embeddings.extend([[0] * 1536] * len(batch))
return all_embeddings

def prepare_items_for_cosmos(items):
    """ 데이터 항목을 Cosmos DB에 저장하기 위해 준비하는 함수

    매개변수:
    items (list): 처리할 데이터 항목 목록

    반환값:
    list: 임베딩과 ID가 추가된 데이터 항목 목록
    """
    # 각 항목에 대해 텍스트 표현 생성
    for item in items:
        item['locationText'] = f"{item['city']} {item['county']}"
        item['textRepresentation'] = solar_data_to_text(item)
        # 고유 ID 생성 (날짜_시간_지역코드 형식)
        item['id'] = f"{item['fcstDate']}_{item['fcstTime']}_{item['regCd']}"

    # 모든 텍스트 표현을 리스트로 추출
    locations = [item['locationText'] for item in items]
    texts = [item['textRepresentation'] for item in items]

    # 배치로 임베딩 생성
    location_embeddings = batch_generate_embeddings(locations)
    embeddings = batch_generate_embeddings(texts)

    for i, location_embedding in enumerate(location_embeddings):

```



```

        items[i]['locationVector'] = location_embedding

# 생성된 임베딩을 각 항목에 추가
for i, embedding in enumerate(embeddings):
    items[i]['contentVector'] = embedding

return items

def bulk_insert_to_cosmos(items, batch_size=100, max_retries=5):
    """ 여러 항목을 Cosmos DB에 일괄 삽입하는 함수

    매개변수:
        items (list): 삽입할 데이터 항목 목록
        batch_size (int): 한 번에 처리할 항목 수(기본값: 100)
        max_retries (int): 실패 시 최대 재시도 횟수(기본값: 5)

    반환값:
        tuple: (성공한 항목 목록, 실패한 항목 목록)
    """
    successful_items = [] # 성공적으로 삽입된 항목 저장
    failed_items = []     # 삽입 실패한 항목 저장

    # 배치 크기로 항목들 나누기
    batches = [items[i:i + batch_size] for i in range(0, len(items), batch_size)]

    for batch_index, batch in enumerate(batches):
        retry_count = 0
        # 최대 재시도 횟수까지 시도
        while retry_count < max_retries:
            try:
                batch_successful = []
                batch_failed = []

                # 배치 내 각 항목 처리
                for item in batch:
                    try:
                        # Cosmos DB에 항목 삽입 또는 업데이트
                        container.upsert_item(body=item)
                        batch_successful.append(item)
                    except CosmosHttpResponseError as e:
                        if e.status_code == 429: # Too Many Requests (요청 제한)
                            # 요청 제한 발생 시 전체 배치 재시도
                            logging.warning(f"Cosmos DB 요청 제한 발생, 재시도 대기 중... (배치 {batch_index})")
                            continue
                        else:
                            batch_failed.append(item)

            except Exception as e:
                logging.error(f"Batch {batch_index} 처리 중 오류 발생: {e}")
                continue

            retry_count += 1

    return (batch_successful, batch_failed)

```

```

        retry_delay = int(e.http_headers.get('x-ms-retry-after-ms', 100
        time.sleep(retry_delay)
        raise e # 예외를 다시 발생시켜 예외 처리로 배치 재시도
    else:
        # 다른 오류는 개별 항목만 실패로 처리
        logging.error(f"아이템 삽입 오류 (ID: {item.get('id')}): {str(e)}")
        batch_failed.append(item)

# 성공 및 실패 항목을 결과 목록에 추가
successful_items.extend(batch_successful)
failed_items.extend(batch_failed)

# 배치 처리 결과 로깅
logging.info(f"배치 {batch_index+1}/{len(batches)} 처리 완료: {len(batch_successful)} 성공, {len(batch_failed)} 실패")

# 현재 배치 처리 완료, 다음 배치로 이동
break

except CosmosHttpResponseError as e:
    # 전체 배치에 대한 오류 (주로 요청 제한)
    retry_count += 1
    # 지수 백오프: 재시도 간격을 점점 늘림 (최대 60초)
    retry_delay = min(2 ** retry_count, 60)
    logging.warning(f"배치 {batch_index+1} 실패, {retry_count}/{max_retries} 재시도
    time.sleep(retry_delay)

except Exception as e:
    # 예상치 못한 기타 오류
    logging.error(f"배치 {batch_index+1} 처리 중 예상치 못한 오류: {str(e)}")
    failed_items.extend(batch)
    break

# 최대 재시도 횟수 초과 시 실패로 처리
if retry_count >= max_retries:
    logging.error(f"배치 {batch_index+1} 최대 재시도 횟수 초과, {len(batch)} 항목 실패")
    failed_items.extend(batch)

return successful_items, failed_items

```

```

app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

@app.route(route="solar_predict_cosmosdb")

```

```

def solar_predict_cosmosdb(req: func.HttpRequest) → func.HttpResponse:
    logging.info('Python HTTP trigger function processed a request.')

    try:
        # 요청 본문(JSON)을 파싱
        req_body = req.get_json()

        # 입력이 배열인 경우(여러 레코드) 또는 단일 레코드인 경우 모두 처리
        items = req_body if isinstance(req_body, list) else [req_body]

        # 데이터 준비 (텍스트 변환 및 임베딩 생성)
        prepared_items = prepare_items_for_cosmos(items)

        # Cosmos DB에 일괄 삽입
        successful_items, failed_items = bulk_insert_to_cosmos(
            prepared_items,
            batch_size=25,
            max_retries=3
        )

        # 처리 결과 생성
        result = {
            "status": "완료" if not failed_items else "부분 완료",
            "total_requested": items,
            "total_processed": len(prepared_items),
            "successful": len(successful_items),
            "failed": len(failed_items)
        }

        # HTTP 200 응답 반환
        return func.HttpResponse(
            json.dumps(result),
            mimetype=APPLICATION_JSON,
            status_code=200
        )
    except Exception as e:
        # 오류 발생 시 HTTP 500 오류 응답 반환
        return func.HttpResponse(
            json.dumps({"status": "error", "message": str(e)}),
            mimetype="application/json",
            status_code=500
        )

```

```

def query_similar_data(user_query, top_k=5):
    """ 사용자 질의와 의미적으로 유사한 태양광 데이터를 검색하는 함수

    매개변수:
        user_query (str): 사용자 질의 텍스트
        top_k (int): 반환할 최대 유사 데이터 수(기본값: 5)

    반환값:
        list: 유사한 데이터 목록
    """
    # 사용자 질의를 벡터(임베딩)로 변환
    embedding_response = openai_client.embeddings.create(
        input=user_query,
        model=openai_embeddings_deployment
    )
    query_vector = embedding_response.data[0].embedding

    # Cosmos DB에서 벡터 유사도 검색 쿼리 실행
    # VectorDistance 함수로 벡터 간 거리(유사도)를 계산하여 정렬
    query = f"""
    SELECT TOP {top_k} c.textRepresentation, c.city, c.county, c.fcstDate,
        c.fcstTime, c.pcap, c.qgen, c.srad, c.temp, c.wspd
    FROM c
    ORDER BY VectorDistance(c.contentVector, {query_vector})
    """

    # 쿼리 실행 및 결과 반환
    results = list(container.query_items(
        query=query,
        enable_cross_partition_query=True
    ))

    return results

def generate_response(user_query):
    """ RAG(Retrieval-Augmented Generation) 패턴으로 사용자 질의에 대한 응답 생성

    매개변수:
        user_query (str): 사용자 질의 텍스트

    반환값:
        str: 생성된 응답 텍스트
    """

```

```

# 1. 관련 데이터 검색 (Retrieval 단계)
context_data = query_similar_data(user_query)

# 2. 검색된 데이터로 컨텍스트 구성
context = "다음은 질문과 관련된 태양광 발전 데이터입니다:\n\n"
for i, data in enumerate(context_data, 1):
    context += f"데이터 {i}: {data['textRepresentation']}\n\n"

# 3. OpenAI 모델을 사용하여 응답 생성 (Generation 단계)
messages = [
    {"role": "system", "content": "당신은 태양광 발전 데이터 분석 전문가입니다. 제공"},
    {"role": "user", "content": f"질문: {user_query}\n\n컨텍스트: {context}"}
]

# 채팅 완성 API 호출
response = openai_client.chat.completions.create(
    model=openai_gpt_model,
    messages=messages,
    temperature=0.3 # 낮은 temperature로 보다 일관된 응답 생성
)

return response.choices[0].message.content

@app.route(route="chat_rag", methods=[func.HttpMethod.POST])
def chat_rag(req: func.HttpRequest) → func.HttpResponse:
    """ RAG 기반 질의응답 처리를 위한 HTTP 엔드포인트

    매개변수:
        req (HttpRequest): HTTP 요청 객체

    반환값:
        HttpResponse: HTTP 응답 객체
    """
    logging.info('Python HTTP trigger function processed a request.')

    # 요청에서 사용자 질문 추출
    req.get_json()
    user_question = req.get_json().get("question")
    logging.info(user_question)
    # RAG 방식으로 응답 생성
    answer = generate_response(user_question)
    logging.info(answer)

```

```
# 응답 반환
return func.HttpResponse(
    json.dumps({"status": "success", "items": answer}),
    mimetype=APPLICATION_JSON,
    status_code=200
)
```

Stream Analytics Query

▼ 코드 (Power BI 및 Cosmos DB 저장)

```
With [solarpredictdata] as (
    SELECT try_cast(pcap as float) "설비용량",
           try_cast(qgen as float) "발전량",
           try_cast(srad as float) "일사량",
           try_cast(temp as float) "기온",
           try_cast(wspd as float) "풍속",
           city "도시",
           county "시군구",
           lat "위도",
           lon "경도",
           regCd "행정코드",
           substring(fcstDate, 1, 4) + '-' + substring(fcstDate, 5, 2) + '-' + substring(fcstDate, 7, 2) "날자",
           substring(fcstTime, 1, 2) "시간"
    FROM [1dt030-solar-hub]
), [processed_data] as (
    SELECT round(try_cast([solarpredictdata].[발전량] / [solarpredictdata].[설비용량]) * 100 as bigint), -1) "이용률",
           ([solarpredictdata].[발전량] / [solarpredictdata].[설비용량]) * 100 "실제이용률",
           *,
           case when try_cast([solarpredictdata].[시간] as bigint) < 12 then '오전'
                else '오후'
           end "오전오후구분",
           CONCAT([solarpredictdata].[날자], '-',
                  [solarpredictdata].[시간], '-',
                  [solarpredictdata].[행정코드]) as id
    FROM [solarpredictdata]
)
SELECT *
```

```

INTO [1dt030-solar-power-bi]
FROM [processed_data]

SELECT try_cast(pcap as float) pcap,
       try_cast(qgen as float) qgen,
       try_cast(srad as float) srad,
       try_cast(temp as float) temp,
       try_cast(wspd as float) wspd,
       city,
       county,
       lat,
       lon,
       regCd,
       substring(fcstDate, 1, 4) + '-' + substring(fcstDate, 5, 2) + '-' + substring(fcstDate, 7, 2) fcstDate,
       substring(fcstTime, 1, 2) fcstTime
INTO [1dt030-solar-cosmosdb]
FROM [1dt030-solar-hub]

```

```

-- PowerBI만 연동
With [solarpredictdata] as (
    SELECT try_cast(pcap as float) "설비용량",
           try_cast(qgen as float) "발전량",
           try_cast(srad as float) "일사량",
           try_cast(temp as float) "기온",
           try_cast(wspd as float) "풍속",
           city "도시",
           county "시군구",
           lat "위도",
           lon "경도",
           regCd "행정코드",
           substring(fcstDate, 1, 4) + '-' + substring(fcstDate, 5, 2) + '-' + substring(fcstDate, 7, 2) "날자",
           substring(fcstTime, 1, 2) "시간"
    FROM [1dt030-solar-hub]
) select round(try_cast([solarpredictdata].[발전량] / [solarpredictdata].[설비용량])
* 100 as bigint), -1) "이용률",
([solarpredictdata].[발전량] / [solarpredictdata].[설비용량]) * 100 "실제이용률",
*,
case
    when try_cast([solarpredictdata].[시간] as bigint) < 12 then '오전'
    else '오후'

```

```
end "오전오후구분"  
INTO  
[1dt030-solar-power-bi]  
FROM  
[solarpredictdata]
```